

PRACTICAL WEEK 2

Task 1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

CODE:

```
179.c: 3:09  File: 264.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int source_fd, destination_fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    // Open source file for reading
    source_fd = open(argv[1], O_RDONLY);
    if (source_fd == -1)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open/create destination file for writing
    destination_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (destination_fd == -1)
    {
        perror("Error opening destination file");
        close(source_fd);
        return 1;
    }

    // Read from source and write to destination
    while ((bytes_read = read(source_fd, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(destination_fd, buffer, bytes_read);
        if (bytes_written != bytes_read)
        {
            perror("Error writing to destination file");
            close(source_fd);
            close(destination_fd);
            return 1;
        }
    }

    if (bytes_read == -1)
    {
        perror("Error reading source file");
    }
    else
    {
        printf("File copied successfully.\n");
    }

    // Close both files
    close(source_fd);
    close(destination_fd);

    return 0;
}
```

```
screenkhilreddy@MacBookPro ~ % gcc 179.c
clang: error: no such file or directory: '179.c'
clang: error: no input files
screenkhilreddy@MacBookPro ~ % nano 264.c
screenkhilreddy@MacBookPro ~ % gcc 264.c -o 264
screenkhilreddy@MacBookPro ~ % ./264
Usage: ./264 <source_file> <destination_file>
screenkhilreddy@MacBookPro ~ %
```

OUTPUT:**Task-2:** Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

```
Last login: Sat Aug 25 11:30:21 on tty800
screen@hlreddy@macbookPro ~ % cat sample.txt
cat sample.txt: No such file or directory
screen@hlreddy@macbookPro ~ % echo "Hello, Operating System!" > sample.txt
$quotes cat sample.txt
$quotes strace cat sample.txt
$quotes cat trace.txt
$quotes evdevctl "cat"/in/cat", [{"cat", "sample.txt"}, ...] = 0
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3
read(3, "Hello, Operating System!\n", 131072) = 24
write(1, "Hello, Operating System!\n", 24) = 24
read(2, "", 131072) = 0
close(3) = 0
exit_group(0) = ?
$quotes █
```

System Call	Purpose	Kernel Service
execve()	Starts the cat program	Process/program execution
openat()	Opens sample.txt	File-system access
read()	Reads file contents	File I/O
write()	Displays contents on terminal	Output/I/O
close()	Closes the file descriptor	Resource management
exit_group()	Terminates the process	Process termination

